

HubIT Maintenance Management System

MMS Technical Architecture & Task Flow Documentation

Author: Antigravity Pair Programmer

Date: June 2026

System Version: 2.1.0-prod

1. System Architecture Overview

The HubIT Maintenance Management System (MMS) is a tenant-aware, real-time application built to schedule, track, and execute physical equipment maintenance. It integrates preventive maintenance routines, spare parts inventories, and IoT device telemetry in a single interface.

1.1 File & Directory Layout

The application is structured according to the clean Model-View-Controller (MVC) design pattern:

- **index.php**: Front controller handling dynamic baseUrl routing and autoloader setup.
- **app/routes.php**: Mapping of HTTP request methods and URIs to Controller Actions.
- **app/controllers/**: Logic processors extracting request parameters and compiling Views.
- **app/models/**: Databases queries, transactions, and business rule evaluations.
- **app/views/**: Interface layout scripts rendering data securely with CSRF tokens.
- **app/middleware/**: Access controllers, authentication checks, and tenant config handlers.

2. Database Schema Details

The system relies on PostgreSQL schemas designed for multi-tenant isolation. The primary data relations include organizations, assets, checklist templates, and routine work orders.

2.1 Assets Configuration Table

Stores the metadata of physical units (machines, sensors) mapped to departments and vendors:

```
CREATE TABLE assets (  
  id SERIAL PRIMARY KEY,  
  tenant_id UUID NOT NULL,  
  asset_id VARCHAR(50) NOT NULL UNIQUE,  
  asset_name VARCHAR(255) NOT NULL,  
  location_id_1 VARCHAR(100),  
  status VARCHAR(20) DEFAULT 'active',  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

2.2 Routine Work Orders Table

Tracks scheduled active and completed maintenance jobs with technician assignments and date windows:

```
CREATE TABLE routine_Work_Orders (  
  id SERIAL PRIMARY KEY,  
  tenant_id UUID NOT NULL,  
  asset_id VARCHAR(50) NOT NULL REFERENCES assets(asset_id),  
  checklist_id VARCHAR(255) NOT NULL,  
  maint_start_date DATE NOT NULL,  
  technician_name VARCHAR(255),  
  status VARCHAR(50) DEFAULT 'Pending'  
);
```

3. Operational Task Flows

The application coordinates several step-by-step processes to generate, assign, and execute maintenance checklists.

3.1 Automated Maintenance Scheduler Flow (TBPM)

The Time-Based Preventive Maintenance (TBPM) scheduler runs in the background to instantiate work orders:

Step	Actor / Component	Action / Process
1	Cron Job / CLI	Triggers tbpm_scheduler.php script periodically.
2	Database Query	Checks assets due for maintenance based on scheduler intervals.
3	RoutineWorkOrderModel	Inserts new work order with status 'Pending' into database.
4	AssetMaintenanceModel	Calculates and stores the next maintenance date, marking tasks as processed.

3.2 Technician Checklist Execution Flow

Once a work order is active, the technician fills out items and submits them:

Step	Actor / Component	Action / Process
1	Technician / Dashboard	Selects an active work order and clicks 'Begin'. Status updates to 'On-Going'.
2	Checklist Controller	Generates checklist instance mapping all templates and tasks in active memory.
3	MaintChecklistUpdateModel	Atomically saves item results and notes inside a secure SQL transaction.
4	Archive / Cleanup Service	On completion, moves records to history tables and deletes active tasks.

4. Security & Automated SSL Updates

To guarantee traffic encryption (HTTPS) across domains, the system implements automated SSL certificate management:

4.1 Chicken-and-Egg Startup Resolution

Normally, Apache will fail to start if SSL certificates are missing. However, Certbot requires Apache to be running on port 80 to verify domain ownership. The custom entrypoint script solves this:

1. **Verification:** The container checks for a Let's Encrypt certificate in `/etc/letsencrypt/live/`.
2. **Self-Signed Fallback:** If missing or self-signed, it generates a dummy certificate automatically.
3. **Bootstrap:** Apache boots safely without crashing.
4. **Verification challenge:** Certbot requests the real certificate via the `--webroot` challenge.
5. **Reload:** Once fetched, the real cert replaces the dummy and Apache reloads gracefully.

4.2 Background Renewal Daemon

A background daemon loop inside the web container checks for certificate expirations every 12 hours. It runs the command:

```
certbot renew --webroot -w /var/www/html --deploy-hook 'apachectl graceful'
```

This checks the expiration and triggers a zero-downtime hot reload only when a certificate is successfully renewed.

4.3 Application Level Security

- **Multi-Tenant Scoping:** All database queries enforce the organization ID context (``tenant_id``) derived from the secure session.
- **CSRF Verification:** Cryptographic tokens validate all state-modifying requests (POST).
- **Secure Sessions:** Session IDs are regenerated on authentication (``session_regenerate_id(true)``) to prevent fixation attacks.